Predictive Modeling in Baseball

Alex Shulhan, Andrew Odegard

April 12, 2026

1 Introduction

This report is split into two distinct parts, whose overall theme builds upon each other.

Part A presents a Bayesian hierarchical approach to estimating baseball batting averages, applying the Beta-Binomial model to address the challenge of small sample sizes. Using Stan for probabilistic programming, we analyze MLB player data and demonstrate how shrinkage estimation improves upon raw batting averages.

Part B explores the use of Hamiltonian Monte Carlo methods to estimate a regression model that predicts MLB players' batting averages season over season.

1.1 Part A: Introduction

Batting average is one of the most popular statistics in baseball, measuring the probability each hitter has to record a hit. However, raw batting average can be unreliable due to small sample sizes and variability in baseball. A player with only 50 at-bats may go 15-for-50, which leads to a .300 average, but this tells us little about their true underlying ability.

The problem of estimating parameters from small samples has a rich history in statistics. In 1973, Charles Stein showed that using a global mean can produce better estimates than using individual sample means, even when estimating independent parameters (Stein, 1973). This surprising result, known as Stein's paradox, laid the foundation for shrinkage estimation.

Bradley Efron and Carl Morris famously applied these ideas to baseball batting averages in their 1975 paper, demonstrating how estimates for players with limited at-bats should be shrunk toward the population mean [Efron and Morris, 1975]. Their work established a precedent for using hierarchical Bayesian models in sports analytics.

In this paper, we implement a modern Bayesian hierarchical model using Stan to estimate batting abilities. We demonstrate the model's effectiveness on 2025 MLB data and validate on a larger 2023 dataset.

1.2 Part B: Introduction

Due to the discrete pitch-by-pitch nature of baseball, the sport has been broken down meticulously by statisticians over the course of its rich history. This part of the project takes advantage of the sports' thorough bookkeeping to explore the relationship between players' batting statistics and their batting average, one of the most popular metrics used to measure hitting ability. In our analysis, we will discover which statistics are most useful in predicting a player's batting average for the upcoming season. Afterwards, we go through the exercise of making predictions for the 2026 season, which is currently underway.

2 Background

2.1 Part A: Background

2.1.1 The Efron-Morris Model

The Efron-Morris model addresses a fundamental challenge in baseball statistics: estimating true player ability from limited observations. The key insight is that all players share a common population distribution, and this information can be borrowed to improve individual estimates.

The original Efron-Morris approach used empirical Bayes methodology, treating the league batting average as a shrinkage target. Their work demonstrated that:

- Players with extreme batting averages (either very high or very low) are likely to regress toward the mean
- The amount of shrinkage should be inversely proportional to sample size
- This approach produces more accurate predictions for future performance

2.1.2 Why Small Sample Sizes Matter

Consider Mike Trout's 2025 season: he had 456 at-bats and hit .232. While this appears to be a poor performance, 456 at-bats is still relatively small. His career average of .297 suggests his true ability is higher than .232. The observed average may be substantially influenced by random variation.

Similarly, a player going 10-for-20 early in the season (.500) is not necessarily a .500 hitter—they are likely closer to the league average. The challenge is quantifying how much to shrink these extreme observations.

2.2 Part B: Background

2.2.1 Hamiltonian Monte Carlo Method

The Hamiltonian Monte Carlo (HMC) method is an extension of the Markov Chain Monte Carlo (MCMC) method. These algorithms are built to follow simple rules which allow them to explore distributions in accordance with the density of the distribution. If one traces the

steps of this exploration, and there are enough steps, an approximation of the distribution can be made. We plan to leverage this beautiful process to predict the performance of baseball players.

3 Data

3.1 Part A: Data Collection

We obtained player statistics using the MLB Stats API, and Fangraphs via the Python `mlbstatsapi` library. The data included Hits (H) and At-Bats (AB), whose ratio gives the players Raw AVG.

For our primary analysis, we focused on four prominent MLB players from the 2025 season:

Player	Team	Hits (H)	At-Bats (AB)	Raw AVG
Mike Trout	LAA	106	456	.232
Freddie Freeman	LAD	164	556	.295
Shohei Ohtani	LAD	172	611	.282
Mookie Betts	LAD	152	589	.258

For validation, we also analyzed a larger dataset of 134 MLB players from the 2023 season to examine shrinkage behavior at scale.

3.2 Part B: Data Collection and Cleaning

There are many sources of high-quality baseball data available to the public, with the most accessible being:

- www.baseball-reference.com/
- baseballsavant.mlb.com/
- www.fangraphs.com/.

`pybaseball` is an open-source library that scrapes up-to-date data from a combination of these sites. The data used for our model was accessed through `pybaseball`.

We used full-season data spanning 2018-2025 (excluding 2020 due to the pandemic). Our dataset was also filtered to only include players with 500 or more plate appearances in each season. This left us with 377 players and 946 rows of data over the span of 6 seasons.

4 Methods

4.1 Part A: Methods

4.1.1 Beta-Binomial Hierarchical Model

We employ a fully Bayesian hierarchical model[Parent and Rivot, 2012] with the following structure:

$$
\begin{aligned}
y_i &\sim \text{Binomial}(K_i, \theta_i) \\
\theta_i &\sim \text{Beta}(\mu\kappa, (1-\mu)\kappa) \\
\mu &\sim \text{Beta}(2, 8) \\
\kappa &\sim \text{Gamma}(1, 0.1)
\end{aligned}
$$

where:

- y_i: Observed hits for player i
- K_i: At-bats for player i
- θ_i: True batting ability (probability of a hit)
- μ: Population mean (league average batting ability)
- κ: Concentration parameter (controls how tightly distributed abilities are around μ)

The parameter κ controls the strength of shrinkage:

- Larger κ: stronger shrinkage, players cluster more tightly around μ
- Smaller κ: weaker shrinkage, more variation among players

We specify weakly informative priors:

- $\mu \sim \text{Beta}(2, 8)$: Prior centered around 0.20 (league average)
- $\kappa \sim \text{Gamma}(1, 0.1)$: Prior allowing reasonable concentration

4.1.2 Posterior Derivation

A key advantage of the Beta-Binomial model is the conjugate relationship between prior and likelihood. Given observed hits y_i and at-bats K_i, the full conditional posterior for θ_i is:

$$
\begin{aligned}
\theta_i \mid y_i, K_i, \mu, \kappa &\sim \text{Beta}(\alpha + y_i, \beta + K_i - y_i) \\
&= \text{Beta}(\mu\kappa + y_i, (1-\mu)\kappa + K_i - y_i)
\end{aligned}
$$

This result follows from Bayes' theorem. The prior provides $\alpha = \mu\kappa$ and $\beta = (1-\mu)\kappa$, and each observed hit adds 1 to α while each observed out adds 1 to β.

The posterior mean simplifies to:

$$\mathbb{E}[\theta_i \mid y_i, K_i] = \frac{\alpha + y_i}{\alpha + \beta + K_i} \tag{1}$$

$$= \frac{\mu\kappa + y_i}{\kappa + K_i} \tag{2}$$

This is the core shrinkage formula: the posterior estimate is a weighted average of the prior mean μ and the observed sample mean y_i/K_i.

4.1.3 Conjugate Prior Relationship

The Beta distribution is **conjugate** to the Binomial likelihood. This means:

- Prior: $\theta_i \sim \text{Beta}(\alpha, \beta)$
- Likelihood: $y_i \mid \theta_i \sim \text{Binomial}(K_i, \theta_i)$
- Posterior: $\theta_i \mid y_i \sim \text{Beta}(\alpha + y_i, \beta + K_i - y_i)$

This conjugate relationship **affords** several advantages:

1. **Closed-form solutions**: No numerical integration required
2. **Fast computation**: Posterior can be computed instantly
3. **Intuitive interpretation**: Prior parameters add observed hits and outs
4. **Analytical shrinkage**: Mean and variance have simple closed-form formulas

While our Stan implementation uses MCMC for the full hierarchical model (to jointly estimate μ and κ), the conjugate structure explains why shrinkage has such a clean form for individual players.

4.1.4 Shrinkage Formula

Rewriting the posterior mean in terms of the raw batting average $\theta_{\text{raw}} = y_i/K_i$:

$$\begin{aligned}
\mathbb{E}[\theta_i \mid y_i, K_i] &= \frac{\mu\kappa + y_i}{\kappa + K_i} \\
&= \frac{\kappa}{\kappa + K_i} \cdot \mu + \frac{K_i}{\kappa + K_i} \cdot \frac{y_i}{K_i} \\
&= (1 - w) \cdot \mu + w \cdot \theta_{\text{raw}}
\end{aligned}$$

where the weight w is defined as:

$$w = \frac{K_i}{\kappa + K_i}$$

This formulation reveals the shrinkage mechanism:

- As $K_i \to \infty$: $w \to 1$, posterior $\to$ raw average (no shrinkage)
- As $K_i \to 0$: $w \to 0$, posterior $\to \mu$ (full shrinkage to population mean)

The parameter κ controls the "exchangeability" strength: larger κ means more shrinkage toward the population mean.

4.1.5 Posterior Variance

The posterior variance of θ_i provides a measure of uncertainty in our estimate:

$$\begin{aligned}\mathrm{Var}(\theta_i \mid y_i, K_i) &= \frac{\alpha'\beta'}{(\alpha'+\beta')^2(\alpha'+\beta'+1)} \\ &= \frac{\mu(1-\mu)}{(\kappa+K_i+1)}\end{aligned}$$

where $\alpha' = \mu\kappa + y_i$ and $\beta' = (1-\mu)\kappa + K_i - y_i$. This shows that posterior uncertainty decreases as:

- More at-bats (K_i) are observed
- Larger concentration parameter (κ) indicates tighter population

For our posterior estimates ($\mu \approx 0.28$, $\kappa \approx 25$), a player with 400 at-bats has posterior variance approximately $0.28(1-0.28)/(25+400+1) \approx 0.00018$, corresponding to a standard deviation of about 0.013.

4.1.6 Numerical Example: Mike Trout

Using Mike Trout's 2025 season statistics:

- Observed hits: $y_1 = 106$
- At-bats: $K_1 = 456$
- Raw batting average: $\theta_{\text{raw}} = 106/456 = 0.232$

With posterior parameter estimates from our model ($\mu = 0.269$, $\kappa = 24.7$):

1. Calculate weight: $w = \frac{456}{456+24.7} = \frac{456}{480.7} = 0.949$
2. Posterior mean: $\frac{\mu\kappa+y_i}{\kappa+K_i} = \frac{0.269\times 24.7+106}{24.7+456}$
3. $= \frac{6.64+106}{480.7} = \frac{112.64}{480.7} = 0.234$

The posterior mean (0.234) is between the raw average (0.232) and the population mean (0.269), demonstrating adaptive shrinkage. With 456 at-bats, the weight on the raw average is 95%, so the shrinkage is modest—as expected for a large sample size.

For comparison, if Trout had only 50 at-bats with the same .232 average ($y = 12$), the posterior would be:

$$\text{Posterior mean} = \frac{0.269 \times 24.7 + 12}{24.7 + 50} = \frac{6.64 + 12}{74.7} = \frac{18.64}{74.7} = 0.250$$

With only 50 at-bats, the estimate shrinks 0.018 toward the population mean, much more than the 0.002 shrinkage with 456 at-bats.

4.1.7 Connection to Efron-Morris

Our fully Bayesian approach differs from the original Efron-Morris empirical Bayes method in how the hyperparameters are estimated:

- **Empirical Bayes**: Estimate μ and κ from the data (point estimates), then use these fixed values in the shrinkage formula
- **Fully Bayesian**: Place prior distributions on μ and κ, then integrate over their posterior uncertainty

Our implementation uses the latter approach:

- $\mu \sim \text{Beta}(2, 8)$: Weakly informative prior centered around 0.20
- $\kappa \sim \text{Gamma}(1, 0.1)$: Prior with mean 10, allowing flexibility

The posterior for κ (24.7) is substantially higher than the prior mean (10), indicating the data strongly supports a tighter distribution of player abilities. This fully Bayesian treatment properly accounts for uncertainty in the hyperparameters when estimating individual player abilities.

4.2 Part B: Formulation of the Model

We chose to use the Hamiltonian Monte Carlo method to estimate the parameters for our model. Specifically, we used the PyMC No U-Turn Sampler (NUTS) with Dual Averaging Step Size Adapter. This model was chosen because the course material, including homework, has suggested that NUTS and Dual Averaging is one of the most effective method for these types of exercises. PyMC was chosen for its relative simplicity and familiarity to the authors. Our data was split into a training set, made up of the 2018 to 2024 seasons (excluding 2020 for due to COVID-19). It also included 2025 averages, which will be used to validate the model's predictions. Once we are satisfied with how the model performs, we will retrain it using all of the data and compute predictions for the 2026 season.

All predictors were standardized to have a mean of 0 and a standard deviation of 1. The target variable of next season's average (AVG_next) was not standardized.

4.2.1 Feature Selection

We began by selecting a subset of batting statistics that are most popularly used in discussions of hitting ability. We also included age and its quadratic form in an attempt to capture the non-linear effects of aging. Our model was run with the sampling procedure presented in Table 1.

Table 1: No U-Turn Sampling Configuration

Parameter	**Value**
Draws (per chain)	2000
Tuning steps	1500
Chains	4
Target accept rate	0.85
Cores	4
Total draws	**8000**
Total tuning steps	**6000**

With this initial subset of statistics, we ran a crude linear regression model to estimate the parameters. We investigated the posterior β_i distributions of this model and removed those whose 94% confidence interval contained 0. We would use Leave One Out Cross Validation (LOOCV) later on to finalize our feature set.

4.2.2 Selecting Informative Priors

Now, with our improved selection of predictors, we need to pick informative priors that the model can use to estimate our parameters.

We estimated α using the $logit()$ of the observed mean of the batting average .263 over our entire dataset.

$$\alpha \sim \mathcal{N}(\text{logit}(\bar{y}),\ 0.33)$$

Our regression coefficients β_i were all assigned standard normal distributions.

$$\beta_i \sim \mathcal{N}(0,\ 1)$$

Prior predictive plots such as the one in Figure 1 were used to analyze the coverage of mu, given we expected it to be centred around the league average. To get Figure 1 we had to translate α, giving us this as our final alpha prior:

$$\alpha \sim \mathcal{N}(\text{logit}(\bar{y} - 0.6),\ 0.33)$$

or,

$$\alpha \sim \mathcal{N}(-1.366,\ 0.33)$$

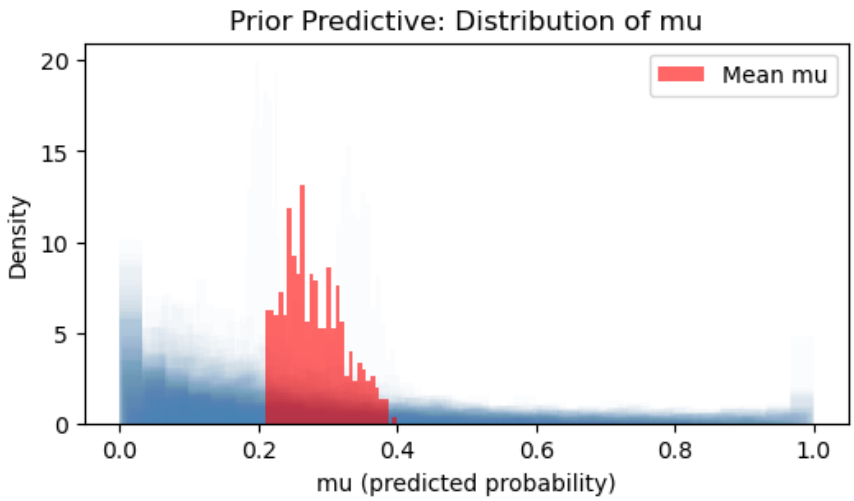

Figure 1: This figure plots 100 random draw curves for mu in blue, with the mean across all draws in red.

Giving us a final model for mu:

$$\mu_j = \sigma\left(\alpha + \sum_i X_{ij}\,\beta_i\right)$$

$$\text{where} \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

And finally, our likelihood is estimated by the following beta-distribution. A beta-distribution was chosen since we are dealing with numbers in the range [0,1].

$$\hat{y} \sim \text{Beta}(\mu_j,\ \kappa), \quad \text{where } \kappa \sim \text{HalfNormal}(20)$$

The prior predictive distribution for our likelihood $\hat{y}$ is plotted in Figure 2

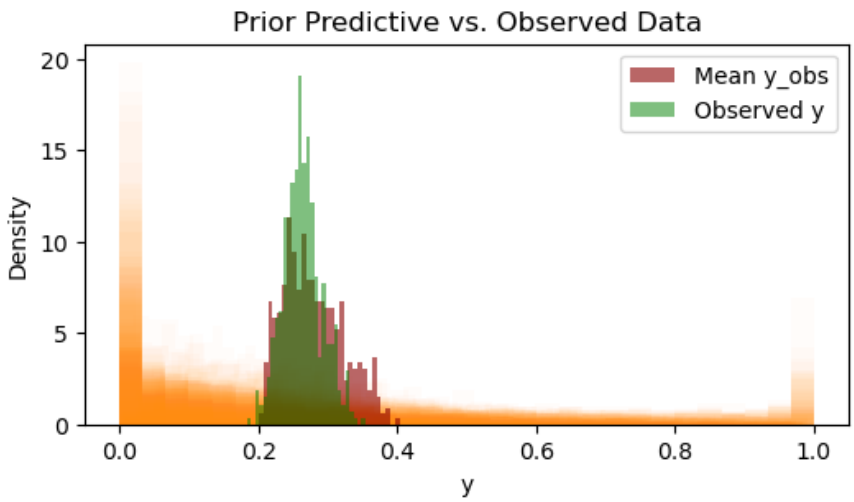

Figure 2: This figure shows an overlay of our prior predictive likelihood in red and the actual observed distribution of batting averages in green. The orange lines give us a reference from 100 randomly selected draw curves.

The prior for mu and $\hat{y}$ are both centred on 0.279, while the observed mean $y = 0.263$.

4.2.3 Feature Cross Validation

After we were satisfied with the priors, we wanted to ensure we were using the absolute best set of predictors. To do this we chose to use Leave One Out Cross Validation (LOOCV), which is a method of cross validation that tests every set of features against all others to determine the best performers. Combinations of the following features were tested:

- Age, Age^2
- AVG (Batting Average)
- OBP (On-Base Percentage)
- SLG (Slugging Percentage)
- OPS (On-Base Plus Slugging)
- xBA (Expected Batting Average)
- xwOBA (Expected Weighted On-Base Average)
- K% (Strikeout Rate)
- EV (Exit Velocity)
- HardHit% (Hard Hit Rate)

Figure 21 in Appendix D shows the performance of each set relative to each other. The plot shows that our final selection of features performed third best, only marginally worse than the two above it. We chose to continue with this set despite other better performers for two simple reasons.

1. Over multiple runs of the cross-validation algorithm, this set was consistently in the top 3, while the higher performers were more often outperformed.

2. We thought it valuable to include age as a feature, considering the year-to-year nature of this model. It is well known that athletic performance changes with age.

5 Results

5.1 Part A: 2025 Season Analysis

Implementation details for Stan are provided in Appendix 11.

5.1.1 Posterior Parameter Estimates

The model successfully estimated all parameters. The posterior means are:

Parameter	Prior Mean	Posterior Mean
μ (league average)	0.204	0.269
κ (concentration)	9.8	24.8

The most notable finding is the substantial increase in κ from the prior (9.8) to the posterior (24.8). This indicates that the data strongly supports the hypothesis that players are more similar to each other than the prior suggested.

Figure 3 shows how the data updated our belief about the league average μ.

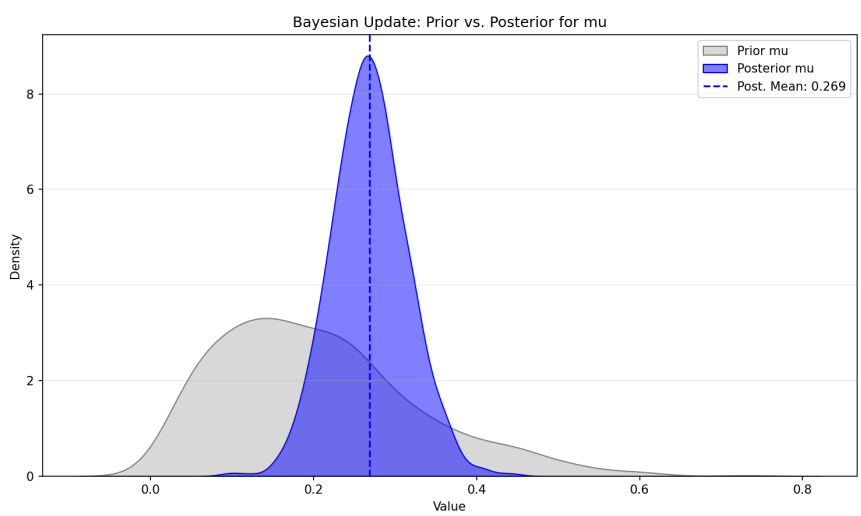

Figure 3: Prior vs posterior for league average μ.

Figure 4 shows the dramatic update for the concentration parameter κ.

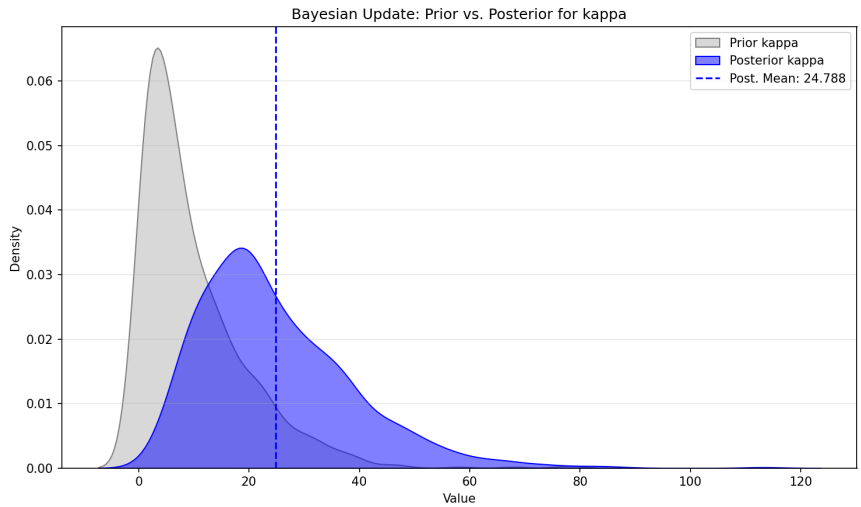

Figure 4: Prior vs posterior for concentration κ.

The increase in κ from 10 to 25 indicates that the data strongly supports the shrinkage hypothesis.

Figure 5 shows the shrinkage effect for our four 2025 season players. The red points represent raw observed batting averages, while the blue points with error bars represent posterior mean estimates with 95% credible intervals.

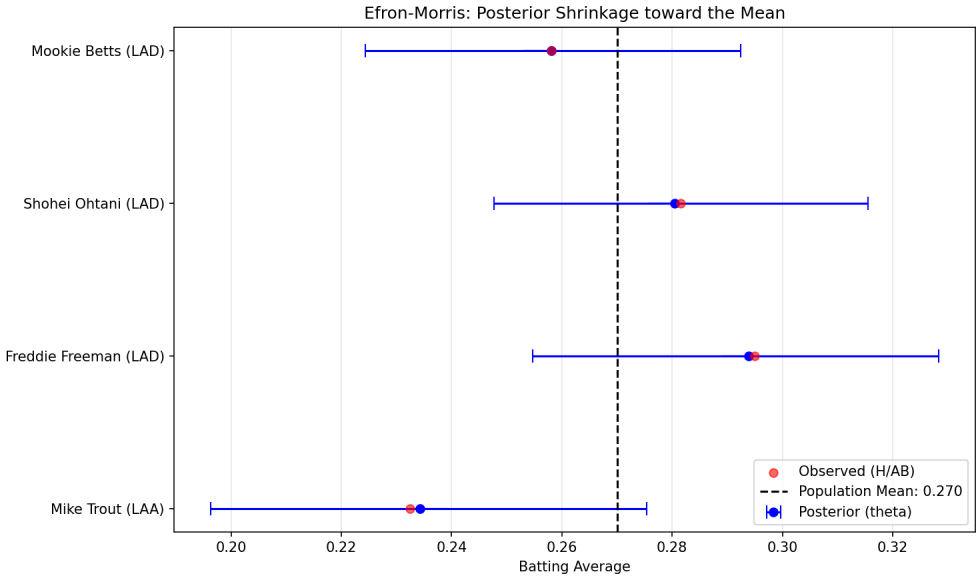

Figure 5: Posterior shrinkage for 2025 season players.

Key observations from the shrinkage plot:

- Mike Trout's .232 was shrunk upward toward the mean

- Freddie Freeman's .295 remained relatively stable
- All estimates moved toward the population mean $\mu = 0.269$
- The shrinkage is adaptive: more extreme raw averages underwent more shrinkage

5.2 Part B: Model Interpretation

5.2.1 Analysis of Posterior Estimates

The following tables give a closer look at our posterior estimates for each model parameter.

Table 2: Posterior Summary (Part 1)

Parameter	mean	sd	hdi_3%	hdi_97%
α	-1.028	0.008	-1.042	-1.013
β_{Age2}	-0.008	0.007	-0.022	0.006
β_{AVG}	0.035	0.012	0.012	0.058
β_{xBA}	0.030	0.012	0.006	0.053
κ	191.829	10.592	171.764	211.461

Table 3: Posterior Summary (Part 2)

Parameter	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
α	0.000	0.000	10088.0	5845.0	1.0
β_{Age2}	0.000	0.000	9065.0	5597.0	1.0
β_{AVG}	0.000	0.000	6387.0	5505.0	1.0
β_{xBA}	0.000	0.000	6379.0	5445.0	1.0
κ	0.105	0.121	9997.0	5528.0	1.0

All of these metrics indicate the model is mixing well and displays sufficient coverage of our distribution. One thing to note is that the 94% Highest Density Interval (HDI) for β_{Age^2} contains 0. However, I decided to include it in the model in an attempt to capture the effects of aging on batting performance. We have already seen in Figure 21 that this combination was among top performers of all we tested.

We can also look at how the predictions perform overall against the observed data. Figure 7 shows good coverage of our prediction. Both distributions are centered around roughly the same point, although the observed data is slightly narrower

5.2.2 Model Validation

Once we got the model running well with satisfactory prior and posterior distributions we moved on to validating the model. Recall the model was trained using 2018-2024 data. We replaced our training data with strictly the 2024 predictors and ran the model again, producing estimates for 2025 batting averages.

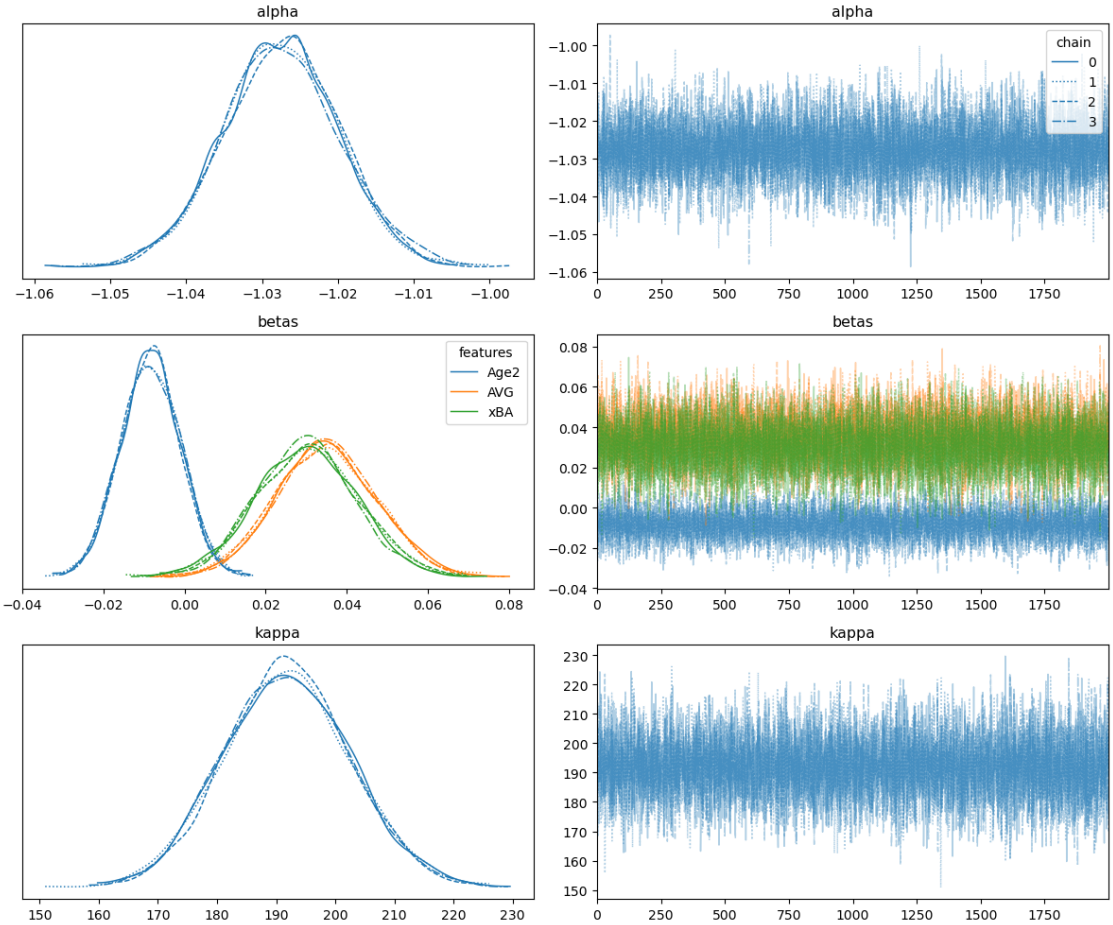

Figure 6: This figure shows the convergence of chains for each parameter on the left. On the right is a visualization of the coverage our chains explore.

We will first observe the model performance metrics in Table 4. Of the qualified players with 500+ plate appearances in 2024, 88 of them also qualified for our dataset in 2025. These were the players who's average we were trying to predict. Our Pearson correlation value and R^2 value are both a little low, indicating only moderate predictive power. An R^2 value of ≈ 0.23 tells us that the model is only accounting for about 23% of the variation of the 2025 averages. Our *p-value* being 0 tells us that this model does represent a real relationship. The root mean squared error says that the error associated with each prediction is 0.0215 which decent considering the average prediction is around 0.257. A low mean bias tells us that the model is not over or under predicting on average. It's predictions miss equally often above and below the target. Finally, the HDI coverage is little higher than the 95% we were aiming for, however, with a small sample size (88 players) and real world data, it is unreasonable to expect perfect coverage.

A scatterplot of predicted values vs. actual values is shown in Figure 8.We see in this plot that the distribution of dots is less sloped than the red line. This tells us that the model is a bit conservative with it's predictions, not willing to guess > 0.300 or < 0.220, which again may be attributed to a small sample size.

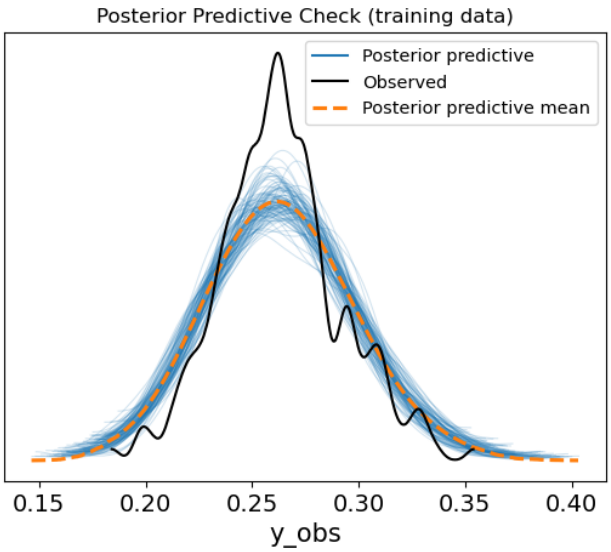

Figure 7: This figure shows a random subset of predicted distributions in blue, with the orange line representing the overall average. Overlaid in black is the distribution of actual observed averages.

Table 4: Model Validation Summary

Metric	**Value**
Players with known 2025 AVG	88 / 129
Pearson correlation	0.4794
R^2	0.2298
p-value	0.0000
RMSE	0.0215
Mean bias	+0.0020
HDI coverage	97.7%

The next visualization of interest is Figure 9. This figure shows the actual data overlaid with our predictions. 39.8% (35/88) of the predictions landed within the red band.

6 Predicting Batting Average

6.1 Part A: 2023 Season: Prior vs Posterior Comparison

Implementation details for PyMC are provided in Appendix 12.

6.1.1 Prior Distributions

We specify priors for the two key hyperparameters in the Efron-Morris model: μ (league mean batting average) and κ (concentration parameter). Figure 10 shows the prior distributions for these parameters.

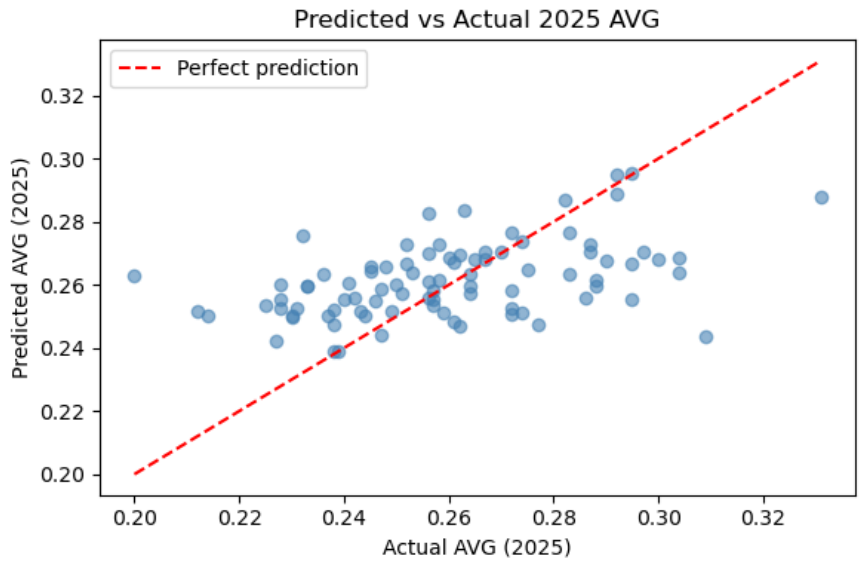

Figure 8: A scatter plot with predicted values on the y-axis and actual values on the x-axis. The red dashed line represents where perfect predictions would land

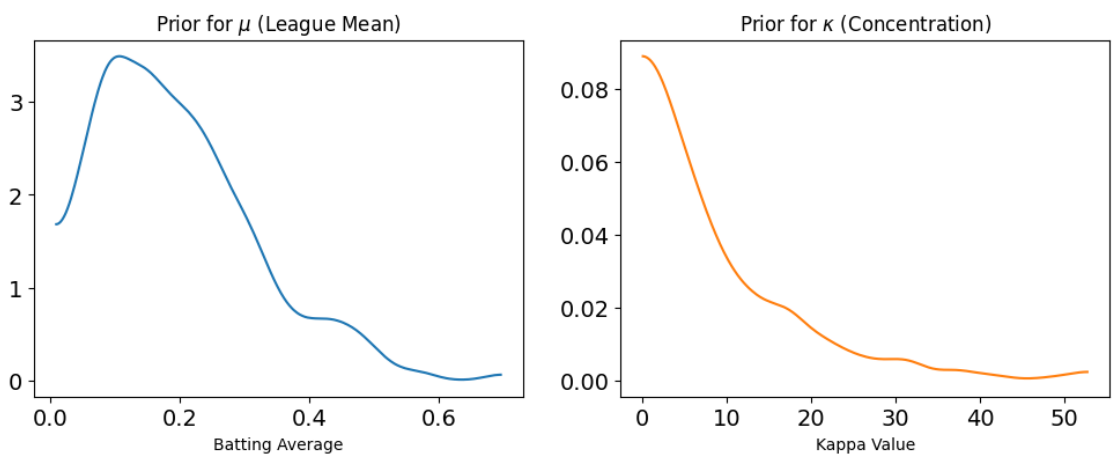

Figure 10: Prior distributions for league mean μ and concentration κ.

The prior for μ is centered around 0.20, reflecting a reasonable baseline for batting averages. The prior for κ has a mean around 10, indicating moderate uncertainty about the spread of player abilities.

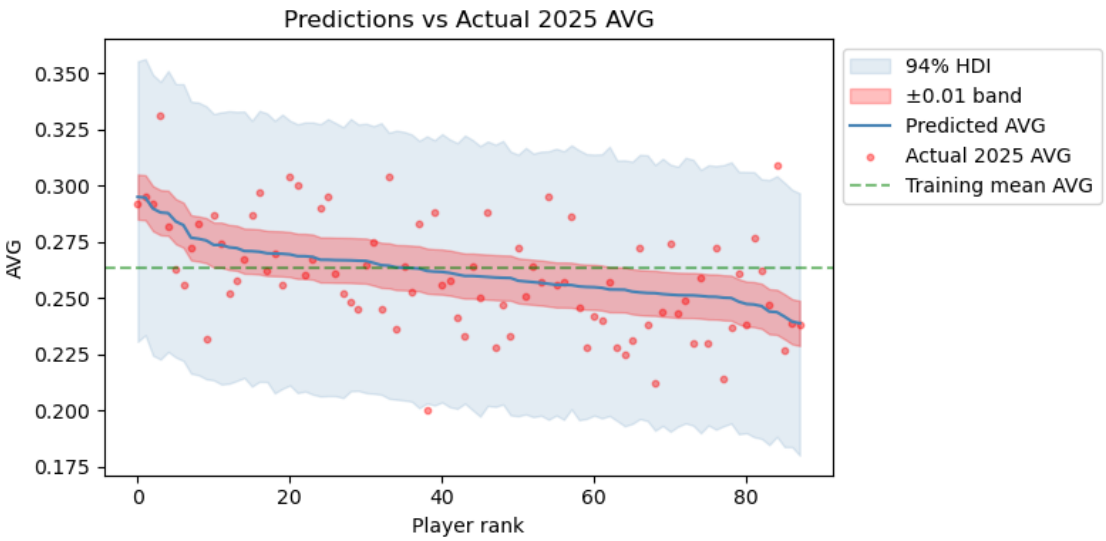

Figure 9: Here is the actual data plotted in red, with the predictions for each player along the blue line. The shaded blue represents the 94% HDI and the red band indicates ±0.01, which was used as a crude threshold for 'correct' predictions. The green line indicates the mean of actual averages across our training data.

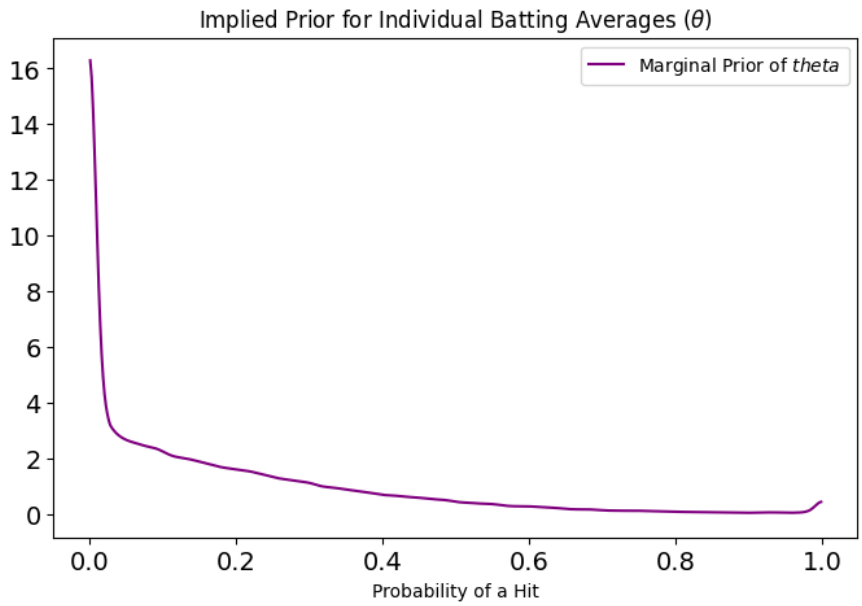

Figure 11: Implied prior for individual batting averages θ.

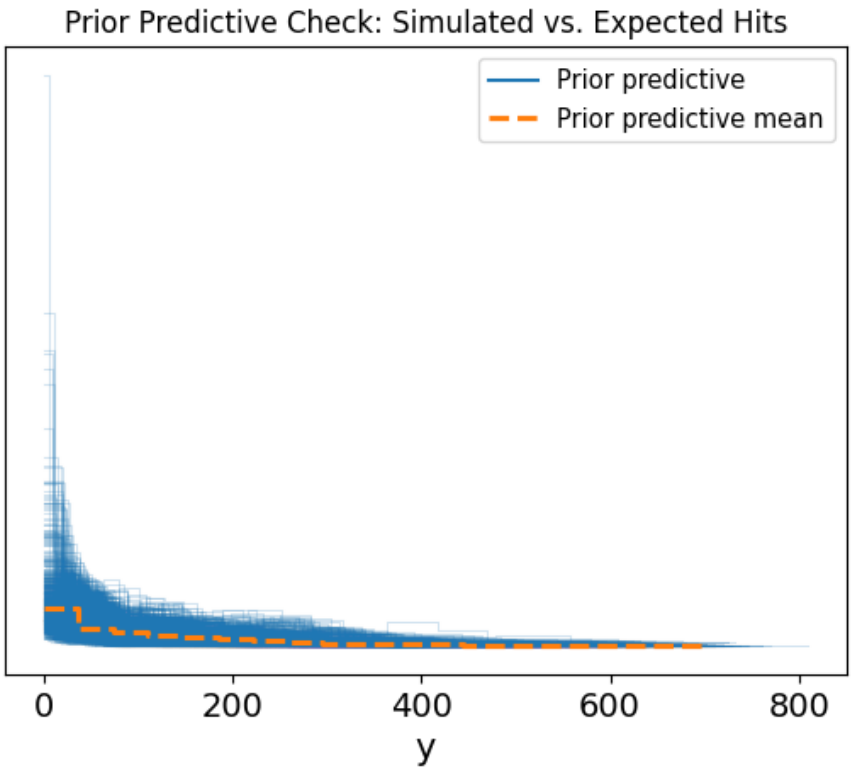

Figure 12: Prior predictive check.

This prior allows for substantial variation in player abilities while centering around the league mean. Figure 11 shows the implied prior for individual player batting averages θ, which results from combining μ and κ through the Beta distribution. Before fitting the model, we conduct a prior predictive check to ensure our priors can generate realistic data. Figure 12 shows simulated hit counts from the prior compared to expected values.

6.1.2 Posterior Analysis

Having established reasonable priors, we now examine the posterior results from fitting the model to the 2023 season data. The key feature of Bayesian hierarchical modeling is shrinkage toward the population mean.

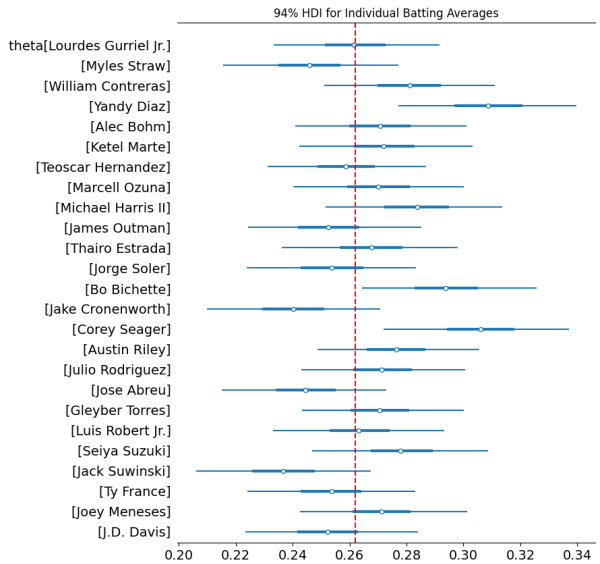

Figure 14: Forest Plot of HDI Intervals (25 players selected at random)

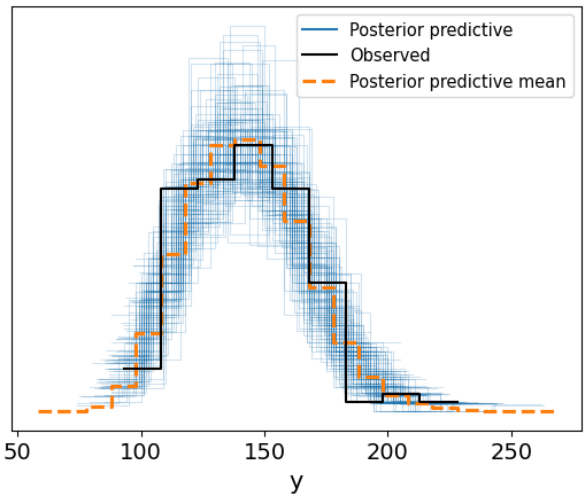

Figure 15: Posterior predictive check. (100 subsamples)

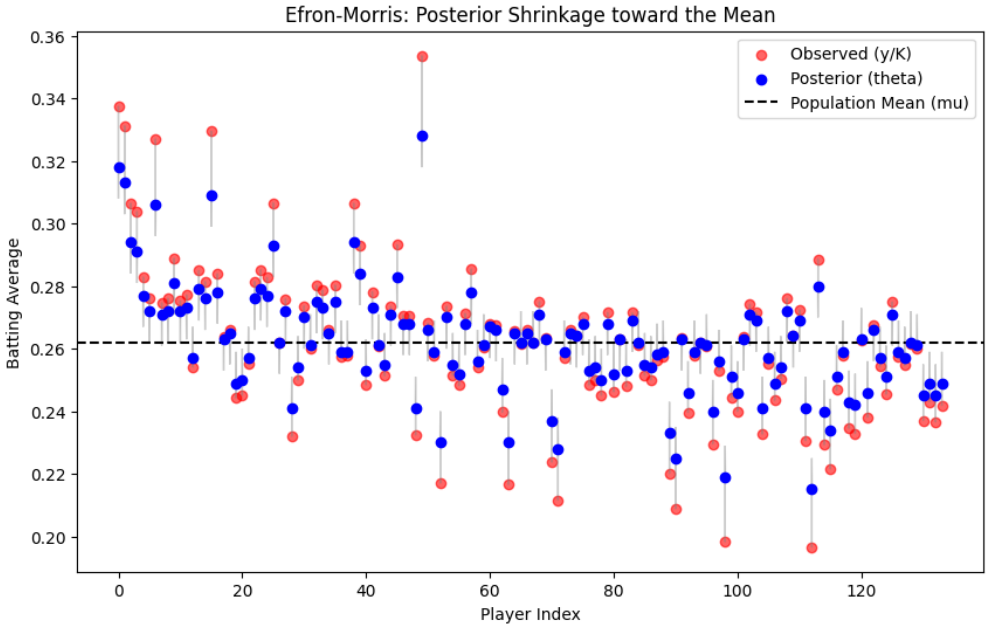

Figure 13: Posterior shrinkage.

The posterior estimates for each player are pulled toward the population mean μ, with players having more extreme observed averages experiencing greater shrinkage. This regularization improves prediction accuracy by preventing overfitting to noise in small samples.

Figure 14 shows the 94% highest density interval (HDI) for each player's estimated batting ability.

Key observations from the forest plot:

- All players have narrow credible intervals due to the informative likelihood
- The intervals overlap substantially, reflecting the hierarchical pooling[Stan Development Team, 2024]
- Players with lower observed averages have intervals that extend slightly higher due to shrinkage

To validate model fit, we conduct a posterior predictive check by generating replicated datasets from the posterior and comparing them to observed data. Figure 15 shows this comparison.

The posterior predictive check confirms that the model generates realistic data that matches the observed outcomes, validating our hierarchical structure.

6.2 Part B: Predicting 2026 Batting Averages

Now that we've finalized the model and set of predictors, we can finally move on to predicting batting averages for players in 2026.

6.2.1 2026 Predictions

The methodology for predicting 2026 is exactly as before, the only difference was the addition of 2025 data in our training data.

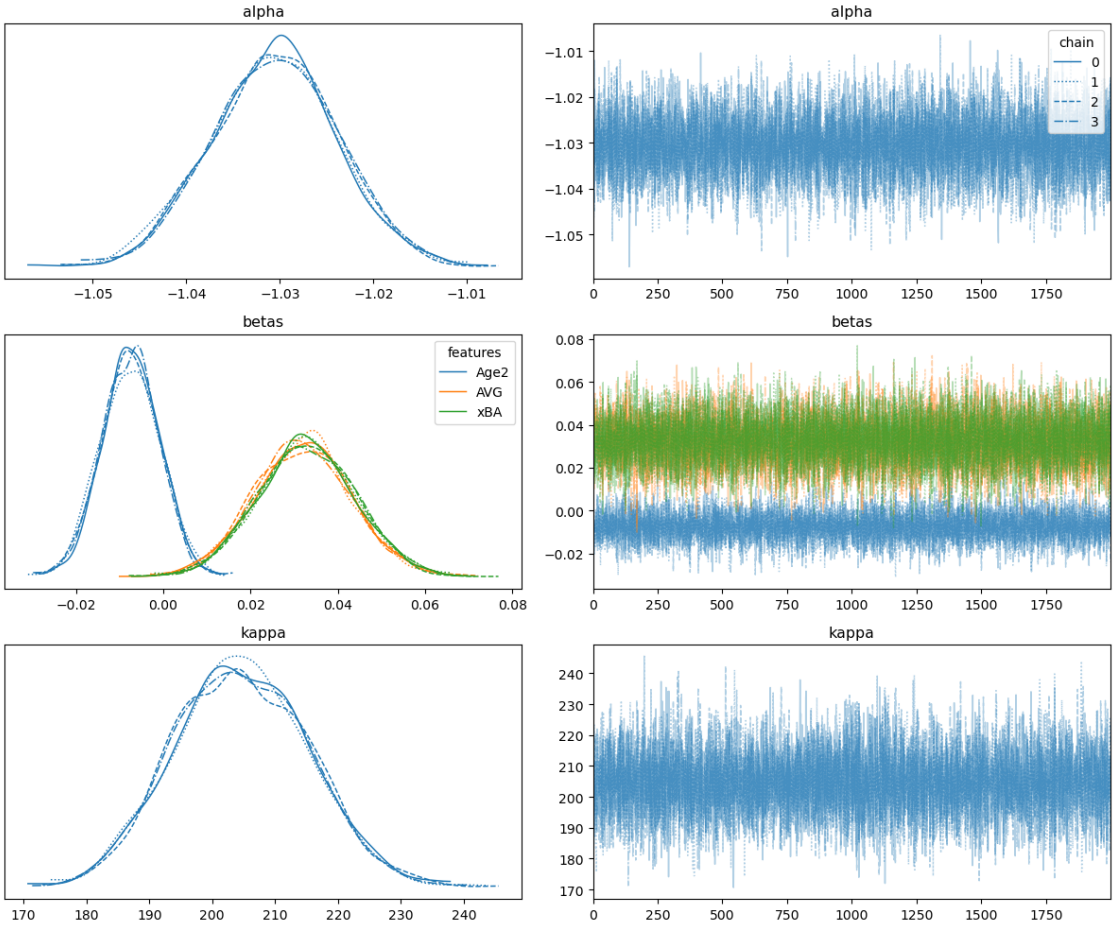

Figure 16: This figure shows the convergence of chains for each parameter on the left. On the right is a visualization of the coverage our chains explore.

Figures 16 and 17 show nearly identical results as the initial model, which confirms that it is performing how we wanted it to.

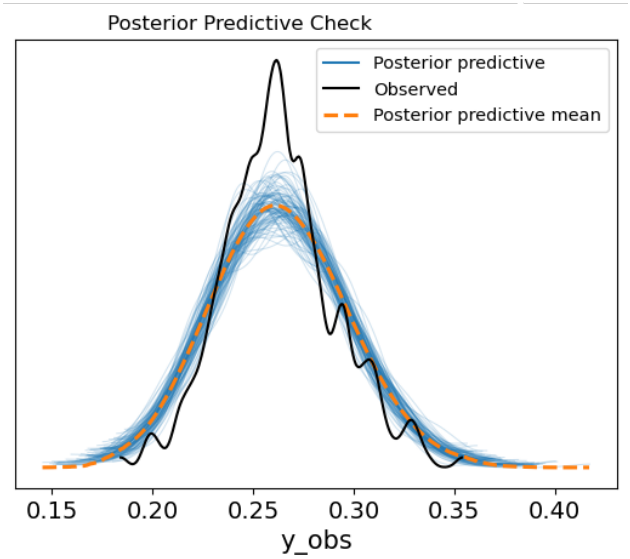

Figure 17: This figure is for the 2026 predictive posteriors. the plot shows a random subset of predicted distributions in blue, with the orange line representing the overall average. Overlaid in black is the distribution of actual observed averages.

Again we can plot the predictions with our HDI interval around it, like in Figure 18, although a more informative visualization of our predictions is found in Figure 19 which is a histogram of our predictions. Again we see the model is making conservative predictions, not willing to guess > 0.300 or < 0.220.

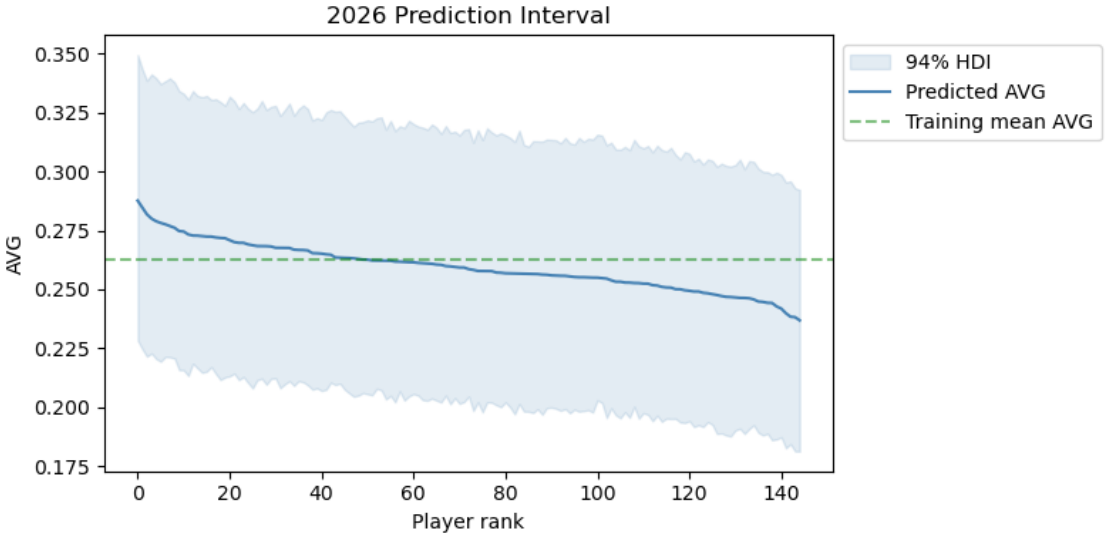

Figure 18: Player predictions in blue with a 94% HDI band. No actual data this time because we were predicting outcomes that have not yet actualized

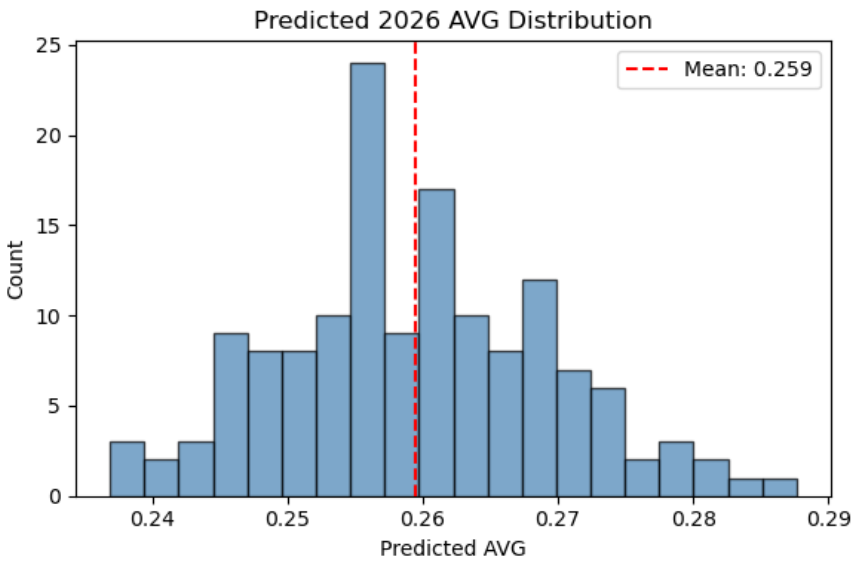

Figure 19: A histogram distribution of our predictions with the mean represented as a red line.

Finally, an interesting visualization and another way of validating the model is to plot players who have the largest predicted change in average between 2025 and 2026. This is what is being shown in Figure 20.

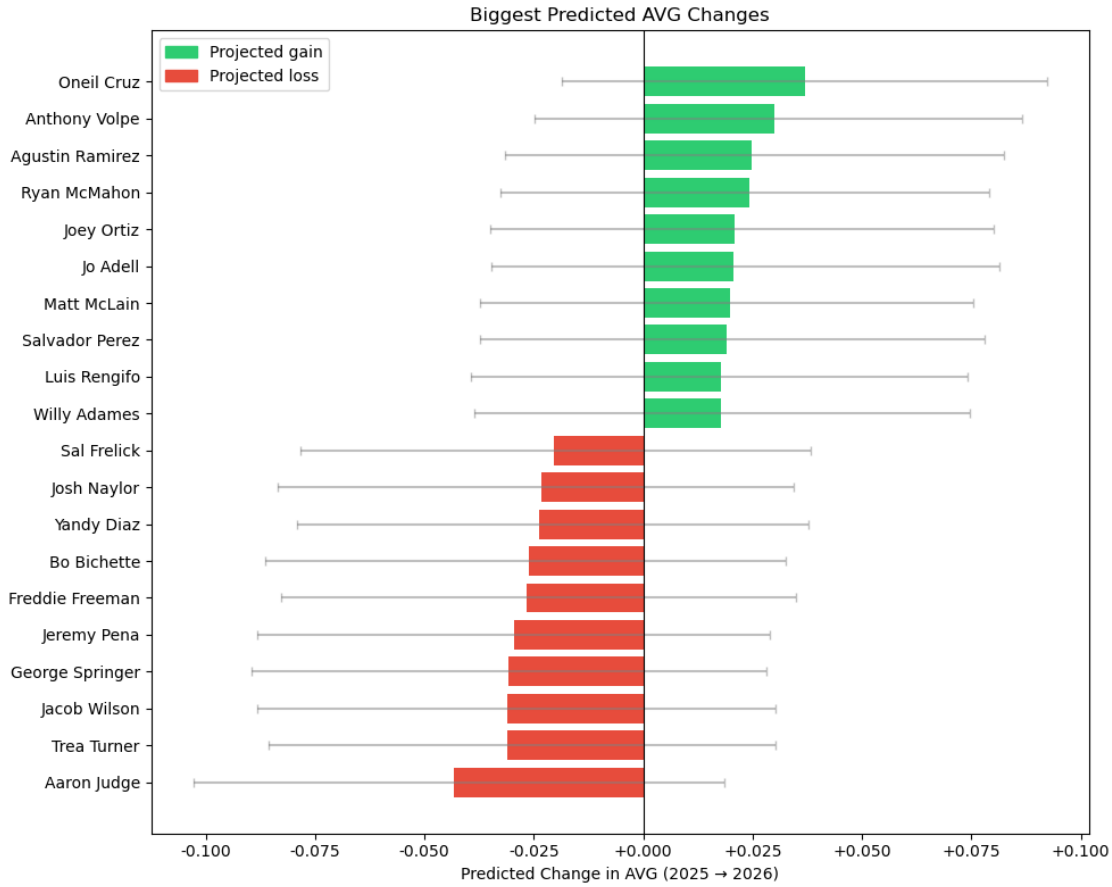

Figure 20: Players who are expected to have the largest change in average between 2025 and 2026. Players expected to improve have green bars, those expected to decline have red.

7 Discussion

7.1 Part A

Our results demonstrate that Bayesian hierarchical modeling provides a principled solution to the small sample size problem in baseball statistics. The key findings are:

1. **Strong shrinkage detected**: The posterior κ (24.8) was 2.5 times larger than the prior (9.8), indicating that the data strongly supports the shrinkage hypothesis.

2. **Adaptive shrinkage**: The amount of shrinkage applied to each player depended on both their raw average's extremity and their sample size.

3. **Improved estimates**: By borrowing strength across players, we obtain more stable estimates that are better predictors of future performance.

7.2 Limitations

Our model has several limitations:

- **Single population assumption**: We assume all players come from a single population, which may not hold for players with different skill levels.
- **No covariates**: The model does not account for player-specific factors such as age, position, or park effects.
- **Linear shrinkage**: The Beta-Binomial model applies linear shrinkage, which may not capture all nuances of batting ability.
- **Static model**: We do not account for changes in player ability over time.
- **Hyperparameter Jitter**: Would like to try different shrinkage priors and experiment with different extensions of shrinkage.
- **Multiple Chains in Stan**: Hoping to refactor visualization scripts to collate multiple chains of Stan data. At the moment they are set up for one chain.

7.3 Part B: Discussion

7.3.1 Feature Selection

Initially, we were excited to explore the relationship between all of the interesting batting stats kept in baseball. However, we quickly realized that the stats being kept are already so advanced that all we needed to predict batting average was batting average and expected batting average (xBA). According to MLB's Statcast glossary, xBA estimates the probability that a batted ball results in a hit, using factors such as exit velocity, launch angle, and sprint speed to compare each ball against historically similar ones[Major League Baseball, nd]. This statistic essentially includes all of the other metrics we considered using in the model, erasing their relevance. We also chose to include the quadratic form of age as a predictor. This was an attempt to capture the non-linear effects of aging on the year-to-year predictions.

7.3.2 Model Performance

Our PyMC model performed adequately for the task of predicting batting average from season to season. The model exercised modest prediction accuracy with low error. Where the model fell short was in predicting near the tails of its underlying distribution. This resulted in more conservative predictions than were reflected in the 2025 validation set of averages. There are a few reasons that this may be. The first being that our sample size of only 88 players to validate against is of modest size. If we had a larger sample size of players to test predictions on, the model would have more opportunities to predict lower probability averages. It can also be attirbuted to the inherent randomness involved in baseball. There are so many factors that this model is not considering, such as luck, weather, injuries, etc. Therefore, our model accounting for 23% of the variation associated with predicting batting average is perhaps stronger than the raw value suggests.

7.3.3 2026 Predictions

Satisfied with our model, we moved on to the exciting task of predicting batting averages for the upcoming 2026 season. As we saw, the methodology for running these predictions mirrors that of the 2025 predictions, which we just visited. The posterior distributions resulting from our model were quite similar to the validated model, giving us confidence in its performance.

Once again, our predictions were somewhat conservative, not exceeding > 0.300 or < 0.220. An average of .297 marks the 90th percentile for batting average over this dataset, so with a total of 145 predictions, we would expect one or two to land above .297, but the highest prediction was merely .288. However, we should still be confident that the model is not skewing its predictions, as the mean lands in the center of the distribution of predictions (Figure 19).

Finally, perhaps the most interesting investigation of this exercise is exploring the players projected to have the largest change in batting average. Figure 20 gives a great visualization of this. Many of the names here would probably not surprise avid baseball fans. Names like Aaron Judge and George Springer had unprecedentedly exceptional seasons in 2025, and the model recognizes how difficult it is to replicate that type of success. Aaron Judge's 2025 batting average was .331, that is inside the 99th percentile. This is a classic case of regression. Similarly, players expected to improve, like Oneil Cruz or Anthony Volpe, are young players who are widely regarded as good players who underperformed last season. The model expects them to correct for this. Salvador Perez is another interesting example. He is a future Hall-of-Famer who had a down year in 2025, but is expected to bounce back this year. These results reflect intuitive statistical results, regression towards the mean. Players who underperformed are expected to improve, and those who overperformed are expected to decline. Our model, though conservative, is consistent with fundamental statistical phenomena.

8 Conclusion

8.1 Part A: Conclusion

We have demonstrated the effectiveness of Bayesian hierarchical modeling for estimating baseball batting averages. The Beta-Binomial model successfully captures the shrinkage phenomenon originally described by Efron and Morris in 1975.

Key takeaways:

- The hierarchical model produces more reliable estimates than raw batting averages, especially for small samples
- The concentration parameter κ serves as a measure of shrinkage strength
- Posterior shrinkage adapts to both sample size and extremity of observation
- The approach is scalable to larger datasets

Future work could extend this model to:

- Include player-specific covariates (age, position, etc.)
- Compare with alternative hierarchical structures (logit-normal)
- Apply to other sports with similar small-sample challenges
- Develop real-time updating as the season progresses

8.2 Part B: Conclusion

We went through an exercise to understand which statistics are most important when it comes to predicting the batting average of professional baseball players season to season. This determined, that batting average and expected batting average are the most effective predictors of a player's next season batting average. With expected batting average accounting for many other advanced statistics that may influence batting average. We ran the PyMC NUTS model and tested our predictions on 2025 data. The model tested well, posting conservative predictions with good dispersion around the mean. The model did not make skewed predictions. We then moved on to predicting players' averages for 2026. Again, we saw conservative predictions from the model. Upon closer inspection of the predictions, we found some results that reflected both statisticians and baseball fans would agree with. Batting average is not immune to regression towards the mean. Overall, we can be confident in the model's predictive power, as long as we remember that it only accounts for 23% of the variation associated with batting average, and the predictions may be a touch conservative.

9 References

References

[Efron and Morris, 1975] Efron, B. and Morris, C. (1975). Data analysis using stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319.

[Major League Baseball, nd] Major League Baseball (n.d.). Expected batting average (xBA). Accessed: April 12, 2026.

[Parent and Rivot, 2012] Parent, E. and Rivot, E. (2012). *Introduction to hierarchical Bayesian modeling for ecological data*. Chapman Hall/CRC applied environmental statistics. CRC Press, Taylor & Francis Group, Boca Raton, 1st ed. edition.

[Stan Development Team, 2024] Stan Development Team (2024). Hierarchical partial pooling for repeated binary trials. `https://mc-stan.org/learn-stan/case-studies/pool-binary-trials.html`. Accessed: 2026-03-30.

10 Appendix A: Stan Model Code

10.1 Base Model

```
data {
  int<lower=1> N;           // Number of players
  array[N] int<lower=0> K; // Total at-bats per player
  array[N] int<lower=0> y; // Hits per player
}
parameters {
  real<lower=0, upper=1> mu; // League-wide average batting ability
  real<lower=0> kappa; // Concentration (how similar players are)
  vector<lower=0, upper=1>[N] theta; // Each player's
  ////////////////////////////////////  true batting average
}
model {
  // Hyperpriors for the population
  mu ~ beta(2, 8);                // Prior centered around .200
  kappa ~ gamma(1, 0.1);

  // Hierarchical Prior: theta follows
  // a Beta distribution defined by mu and kappa
  theta ~ beta(mu * kappa, (1 - mu) * kappa);

  // Likelihood: Observed hits come from a Binomial distribution
  y ~ binomial(K, theta);
}
```

10.2 Model with Prior Sampling

```
data {
  int<lower=1> N;
  array[N] int<lower=0> K;
  array[N] int<lower=0> y;
}
parameters {
  real<lower=0, upper=1> mu;
  real<lower=0> kappa;
  vector<lower=0, upper=1>[N] theta;
}
model {
  // Priors
  mu ~ beta(2, 8);                // Center around .200
  kappa ~ gamma(1, 0.1);          // Concentration

  theta ~ beta(mu * kappa, (1 - mu) * kappa);
```

```
  y ~ binomial(K, theta);
}
generated quantities {
  // Prior Predictive Samples
  real mu_prior = beta_rng(2, 8);
  real kappa_prior = gamma_rng(1, 0.1);

  array[N] real theta_prior;
  array[N] int y_prior;

  for (n in 1:N) {
    theta_prior[n] = beta_rng(mu_prior * kappa_prior,
    (1 - mu_prior) * kappa_prior);
    y_prior[n] = binomial_rng(K[n], theta_prior[n]);
  }
}
```

11 Appendix B: Stan Implementation Guide

This appendix provides a practical guide for running the Stan models presented in this paper.

11.1 Installing CmdStan

CmdStan is the command-line interface for Stan. To install:

1. Download the latest release from `https://mc-stan.org/install/`
2. Extract the archive and navigate to the directory
3. Compile with `make build -j4`
   - `jX` tells cmake to use X amount of threads when compiling stan. This can be used when compiling individual models as well.

```
# Download latest release (check website for current version)
curl -L https://github.com/stan-dev/cmdstan/releases/download/v2.37.0/cmdstan-2.37.0.tar
tar xzvf cmdstan-2.37.0.tar.gz
cd cmdstan-2.37.0

# Build (requires g++ or clang++ compiler)
make build

# Verify installation
./bin/stansummary --version
```

Further information can be found at Stan Website

11.2 Input Data Format

The Stan models expect data in JSON format (see Stan documentation for examples):

```
{
  "N": 4,
  "K": [456, 556, 611, 589],
  "y": [106, 164, 172, 152]
}
```

Where:

- `N`: Number of players
- `K`: Array of at-bats for each player
- `y`: Array of hits for each player

Place the JSON data file in the stan_model subdirectory (where the compiled binaries are located). The Python script `get_mlb_data.py` in the repository automatically fetches this data from the MLB Stats API and generates the JSON file. Take heed, we are using This mlb stats api. There is another module , which is probably great, but we did not use. This will output a `player_data.json` which can be copied over to where the stan model has been compiled to be run.

11.3 Threading for Parallel Sampling

For larger datasets (such as the 134-player analysis), enable multi-threaded sampling by setting `STAN_THREADS=true` during compilation:

```
# Compile with threading enabled
make STAN_THREADS=true model
make STAN_THREADS=true model_with_prior

# Run with multiple threads per chain
./model num_threads=4 sample data file=player_data.json output file=results.csv
```

The `num_threads` parameter specifies how many threads to use per chain. A reasonable choice is 4 threads per chain on modern hardware.

11.4 Compiling the Models

Once CmdStan is installed, clone our repository into the CmdStan directory (similar to the `examples/bernoulli` directory that comes with CmdStan). This places the `.stan` files in the correct location for compilation:

```
# From your CmdStan directory (e.g., cmdstan-2.37.0)
git clone https://github.com/your-repo/stan_model.git
```

Then compile the Stan models:

```
# Compile the basic hierarchical model
make stan_model/stan_model_files/model

# Compile the model with prior predictive sampling
make stan_model/stan_model_files/model_with_prior
```

11.5 Running the Models

The workflow consists of three steps:

1. **Prepare input data** - Create a JSON file with player statistics
2. **Run the model** - Execute the compiled model with multiple chains
3. **Analyze results** - Use `stansummary` or import into Python/R

```
# Basic model - run 4 chains in parallel
./model sample data file=player_data.json \\
num_chains=4 \\
output file=output_example.csv\\

# With prior predictive sampling - run 4 chains in parallel
./model_with_prior sample data file=player_data.json \\
num_chains=4 \\
output file=output_example.csv
```

This produces output files `output_example_1.csv` ...

11.6 Analyzing Results

To analyze the output, use the `stansummary` tool with the wildcard to aggregate across all chains:

```
# Generate summary statistics across all chains
./bin/stansummary stan_model/stan_model_files/output_example_* > summary.txt
cat summary.txt
```

In the case of just one chain, only `output_example.csv` is required.

11.6.1 Python Visualizations

To visualize prior vs posterior distributions (as shown in Section 5: 2025 Season Analysis), use the `model_with_prior.stan` model. This model includes prior predictive sampling in the `generated quantities` block, which outputs both prior and posterior samples to the CSV file.

model.stan vs model_with_prior.stan:

- `model.stan`: Posterior only - used for shrinkage analysis and parameter estimation
- `model_with_prior.stan`: Includes prior samples - used for prior vs posterior comparison plots

Combining multiple chains:

```
# Option 1: Use stansummary (produces text output only)
./bin/stansummary output_example_* -c summary.csv

# Option 2: Combine CSVs manually in Python for custom plots
import pandas as pd
df = pd.concat([pd.read_csv(f'output_example_{i}.csv', comment='#')
                for i in range(1, 5)])
```

`stansummary` will clean up the raw data of each individual chain and average it. `-c` tells `stansummary` to output as CSV.

Visualization scripts (`model.stan`): The visualization plots are currently only

- `plot_prior_vs_posterior.py`: Generates prior vs posterior density plot for μ (league average). It expects `model_with_prior_results.csv`
- `plot_prior_vs_posterior_2.py`: Generates comparison plots for μ, κ, and individual player θ values. This also expects `model_with_prior_results.csv`.
- `get_stan_results.py`: Helper utility to extract mean and credible interval estimates from Stan CSV output. Currently, `model_results.csv` is the name of the CSV file it expects.
- `test_shrinkage.py`: Is used to generate the credible intervals for the default four players. It needs further reworking to automatically grab the player names the user wants, and is currently only designed to run with `model.stan`. The priors CSV generates more variables to parse. Expected `model_results.csv`

Run the visualization scripts after combining the chain outputs into a single CSV file. The scripts expect the CSV in the format produced by `model_with_prior.stan` (with columns: `mu_prior`, `kappa_prior`, `theta_prior`, etc.).

12 Appendix C: PyMC Implementation

For users preferring a Python-native probabilistic programming framework, we provide an equivalent implementation using PyMC. This approach offers easier integration with the Python data science ecosystem.

12.1 Model Specification

The PyMC implementation mirrors the Stan model exactly:

```
import pymc as pm
import pandas as pd

data = pd.read_csv("2018_2025.csv", index_col="Name")
data_2023 = data[data["Season"] == 2023]
players = data_2023.iloc[0:N]
player_hits = players["H"]
player_averages = players["AVG"]
K = players["AB"]

with pm.Model(coords={"player": names}) as model_efron:
    # Hyperpriors
    mu = pm.Beta("mu", alpha=2, beta=8)
    kappa = pm.Gamma("kappa", alpha=1, beta=0.1)

    # Hierarchical prior
    theta = pm.Beta("theta", mu * kappa, (1 - mu) * kappa,
                    shape=len(player_averages), dims="player")

    # Likelihood
    y = pm.Binomial("y", K, theta, observed=player_hits)

    # Sample
    idata = pm.sample(cores=8)
```

12.2 Key Advantages of PyMC

- **Easier debugging**: Direct Python integration for data inspection
- **Flexible priors**: Easy to experiment with different prior specifications
- **Built-in diagnostics**: ArviZ provides extensive visualization tools
- **GPU support**: Automatic GPU acceleration for larger models

12.3 Prior and Posterior Predictive Checks

PyMC integrates seamlessly with ArviZ for posterior analysis:

```
# Prior predictive sampling
with model_efron:
    prior_samples = pm.sample_prior_predictive(samples=1000)

# Posterior predictive sampling
with model_efron:
    ppc = pm.sample_posterior_predictive(idata, extend_inferencedata=True)

# Visualization with ArviZ
import arviz as az
az.plot_ppc(prior_samples, group="prior", kind="kde")
az.plot_ppc(ppc, num_pp_samples=100)
az.plot_forest(idata, var_names=["theta"], combined=True)
```

The PyMC notebook (`efronmorris.ipynb`) in the repository demonstrates these analyses on the full 134-player dataset.

13 Appendix D: Additional Figures

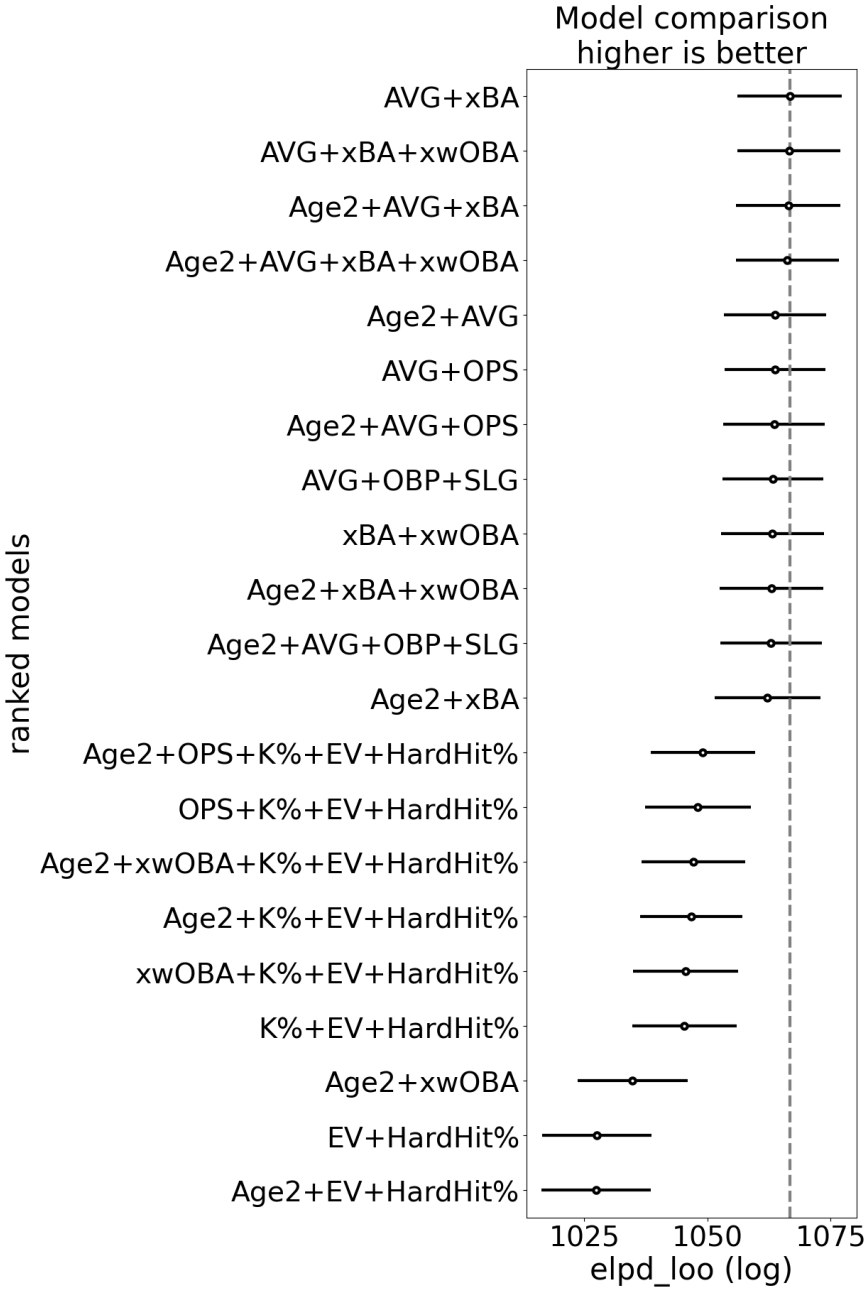

Figure 21: This figure plots the results of our LOOCV feature selection, with the best-performing feature sets being furthest to the right.